Trabajo Práctico Grupal —
Sistema de guía de apoyo para drones

Universidad Tecnológica Nacional — Facultad Regional Buenos Aires

Algoritmos y Estructuras de Datos · Trabajo práctico grupal

Docente: Ing. Mendez

Curso: *K1054*

Fecha de entrega: Nov 2, 2026

Repositorio GitHub: *https://github.com/maurogramajofrba/tp_algoritmos_2026*

Legajo	Nombre	Apellido	Correo
232.954-2	Martín	de Llano Morales	mdellano@frba.utn.edu.ar
1437045	Mauro Adrián	Gramajo	maurogramajo@frba.utn.edu.ar
203.753-1	P. Andrés	Krasnokuki Rousseau	pkrasnokukirousseau@frba.utn.edu.ar
234.666-7	Matias Fernando	Escobar mayta	mfescobar@frba.utn.edu.ar

Descripción de la solución

El programa, escrito en C++, muestra un menú con las siete operaciones pedidas y trabaja sobre una tabla de 200 x 200 órdenes en memoria y sobre archivos binarios de registros. Se compila con `g++ *.cpp -o tp_ejecutable` (requiere C++17 o superior, porque usa `<filesystem>`) y se ejecuta con `./tp_ejecutable`.

Estructuras de datos

Todas están en `tp_types.h`.

Estructura	Uso
`tOrden`	Una celda de la tabla: espera, despegue, aterrizaje, granada 1, granada 2, ataque kamikaze y la siguiente posición (x, y).
`tOrdenArchivo`	Un registro del archivo binario: los mismos campos que `tOrden` más la coordenada (x, y) de la celda.
`tMapaAtaque`	La memoria del dron: una matriz `tOrden ordenes[200][200]`, indexada como `ordenes[x][y]`.
`tChecksAtaque`	Estado de la creación de un ataque (operación 3): acciones ya usadas y posiciones visitadas, guardadas ordenadas para buscarlas con búsqueda binaria.

La tabla ocupa unos 800 KB, por eso se reserva en memoria dinámica con `new` al cargar un archivo y se libera con `delete` al cargar otro o al salir del programa.

Organización del código

Cada operación está en su propio archivo `.cpp` y su prototipo en `tp_lib.h`. `main.cpp` tiene el menú (un `switch` dentro de un `do-while`) y el puntero global a la tabla. `funciones_comunes.cpp` agrupa las funciones que usan todas las pantallas: `ImprimirSeparador` y `EnterParaContinuar`.

Operaciones del menú

1. **Cargar archivo de ataque en memoria** (`cargar_ataque_en_memoria.cpp`). Pide la ruta y lee el archivo dos veces. En la primera pasada valida: coordenadas dentro de la tabla, ninguna celda repetida, un único despegue sin ataques y exactamente una orden final (aterrizaje o ataque kamikaze). Si algo falla, avisa y no carga nada. En la segunda pasada ubica cada registro en su celda.
2. **Mostrar ataque cargado** (`mostrar_ataque_en_memoria.cpp`). Busca la celda de despegue y recorre la ruta siguiendo `siguienteX` y `siguienteY` hasta la orden final. Muestra 15 órdenes por página y espera ENTER para seguir.
3. **Crear un archivo de ataque nuevo** (`crear_nuevo_ataque.cpp`). Pide la ruta, crea el archivo y guía al usuario orden por orden con un submenú: esperar, moverse, soltar granada 1 o 2, aterrizar, ataque kamikaze, cancelar o finalizar la orden. La primera orden es siempre el despegue. Cada orden terminada se graba con `fwrite`; si el usuario cancela, el archivo se borra.
4. **Corregir un registro del archivo** (`corregir_registro_de_archivo.cpp`). Abre el archivo en modo `rb+`, busca en forma secuencial el registro con la x e y indicadas, lo muestra y pide los

datos nuevos. Después retrocede un registro con `fseek` y lo sobrescribe en el mismo lugar con `fwrite`.

5. **Corregir un registro en memoria** (`corregir_registro_de_memoria.cpp`). Hace lo mismo que la operación 4 pero sobre la tabla. Si la orden corregida pasa a ser final, borra de la tabla el resto de la ruta anterior.
6. **Guardar memoria en un archivo nuevo** (`guardar_ataque_en_memoria.cpp`). Pide nombre y carpeta, recorre la ruta desde el despegue y graba un registro por celda, en orden de ejecución.
7. **Visualizar un archivo de ataque en HTML** (`visualizar_ataque_en_html.cpp`). Genera un `.html` con el mismo nombre que el archivo cargado, con una tabla que tiene una fila por orden en el orden de la ruta: coordenada, acciones, espera y siguiente posición.

La opción 0 termina el programa y libera la tabla.

Hipótesis

La solución se apoya en estas hipótesis sobre la tabla, las órdenes y los archivos.

Tabla y coordenadas

- Las coordenadas x e y van de 0 a 199, y la tabla en memoria se indexa como `ordenes[x][y]`.
- Una celda vacía tiene todos sus campos en 0 o `false`.
- La siguiente posición (0, 0) significa “sin siguiente”, porque es el valor inicial de la estructura. Por eso, al crear un ataque (operación 3), una orden de movimiento solo se graba si su siguiente x y su siguiente y son distintas de 0.
- Las celdas aledañas son las 8 vecinas, incluidas las diagonales. Al crear un ataque solo se puede avanzar a una celda aledaña que todavía no se haya usado.

Órdenes

- El despegue puede tener una espera y un movimiento, pero no ataques ni aterrizaje.
- El dron lleva una granada de cada tipo: al crear un ataque, cada una se puede soltar una sola vez.
- El aterrizaje y el ataque kamikaze terminan la ruta. Esas órdenes no tienen siguiente posición, y al cargar un archivo su siguiente se ignora.
- Si una celda tiene varias acciones, se muestran en el orden de precedencia de la consigna: despegue, aterrizaje, granada 1, granada 2, ataque kamikaze, espera y movimiento.
- La espera se expresa en segundos enteros (`unsigned int`).

Archivos

- Un archivo de ataque es binario: una sucesión de registros `tOrdenArchivo`, sin encabezado.
- Las operaciones 3 y 6 graban los registros en orden de ejecución. La operación 1 no depende de ese orden, porque ubica cada registro por su coordenada.
- Al corregir un registro (operaciones 4 y 5) no se cambia su coordenada, y los datos nuevos no se validan al guardarlos: la validación se hace al cargar el archivo con la operación 1.
- La operación 6 guarda solo las celdas de la ruta, empezando por el despegue.
- La operación 7 crea el HTML junto al archivo cargado, con el mismo nombre y extensión `.html`, y lo reemplaza si ya existe.

Diagrama de bloques de subprogramas

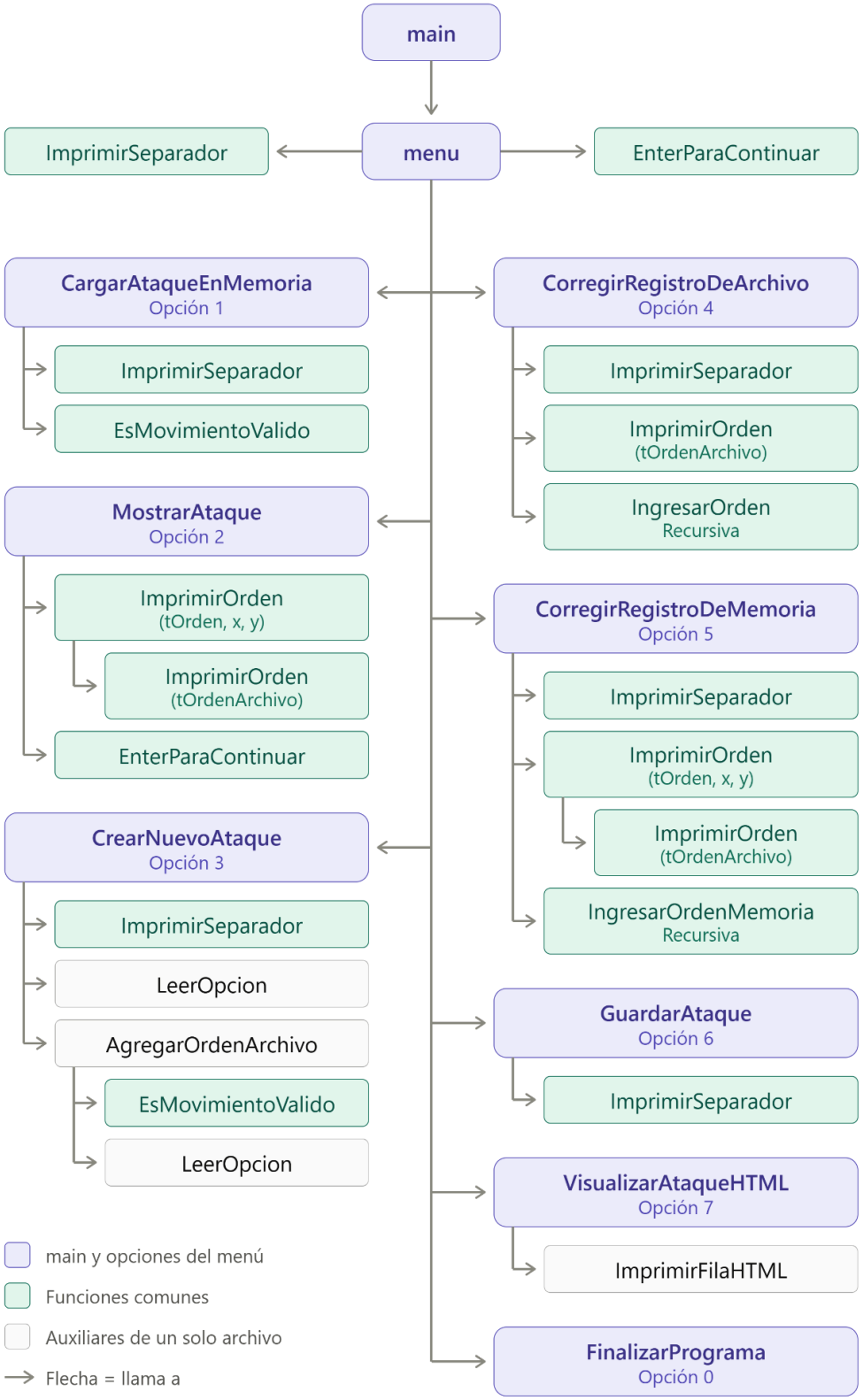


diagrama de bloques · main, menú, 8 opciones y subprogramas auxiliares

El menú se repite hasta que se elige 0.

División de tareas

Cada operación quedó en su propio archivo para que cada integrante pudiera trabajarla por separado en el repositorio.

Parte	Archivos	Integrante
Menú principal y funciones comunes	`main.cpp`, `funciones_comunes.cpp`, `tp_types.h`, `tp_lib.h`	Mauro Adrián Gramajo
1. Cargar archivo de ataque en memoria	`cargar_ataque_en_memoria.cpp`	Martín de Llano
2. Mostrar ataque cargado	`mostrar_ataque_en_memoria.cpp`	Matias Escobar Mayta
3. Crear un archivo de ataque nuevo	`crear_nuevo_ataque.cpp`	Mauro Adrián Gramajo
4. Corregir un registro del archivo	`corregir_registro_de_archivo.cpp`	P. Andrés Krasnokuki
5. Corregir un registro en memoria	`corregir_registro_de_memoria.cpp`	Matias Escobar Mayta
6. Guardar memoria en un archivo nuevo	`guardar_ataque_en_memoria.cpp`	Martín de Llano
7. Visualizar un archivo de ataque en HTML	`visualizar_ataque_en_html.cpp`	Mauro Adrián Gramajo
Informe y casos de prueba	Este documento	Martín de Llano / Mauro Adrián Gramajo

Falta completar quién hizo cada parte.

Casos de prueba

Los 11 casos de prueba dieron el resultado esperado. Se ejecutaron en una misma sesión del programa compilado con `g++` en Linux, y todos los archivos de prueba se crearon con el mismo programa.

N°	Operación	Prueba	Resultado obtenido
1	Menú	Iniciar el programa	Muestra las 7 operaciones y la opción 0
2	3	Crear `ataque_demo.dat`: despegue en (2, 10), espera de 2 s, granada 1, granada 2 y ataque kamikaze en (6, 6)	Archivo creado con 6 órdenes
3	1	Cargar `ataque_demo.dat`	Ataque cargado en memoria
4	2	Mostrar el ataque cargado	Las 6 órdenes en el orden de la ruta
5	4	Corregir (4, 8) en el archivo: agregar una espera de 3 s	Registro reemplazado; al recargar se ve el cambio
6	4	Corregir (50, 50), que no está en el archivo	Aviso y ningún cambio
7	5	Corregir (2, 9) en memoria: espera de 2 s a 5 s	El cambio aparece con la operación 2
8	6	Guardar la memoria en `ataque_corregido.dat`	Archivo creado
9	7	Generar el HTML del ataque cargado	`ataque_demo.html` con una fila por orden
10	4 y 1	Agregar un segundo despegue a `ataque_corregido.dat` y cargarlo	El programa rechaza el archivo
11	1 y 2	Mostrar una ruta de 20 órdenes	Pausa después de 15 órdenes y muestra el resto con ENTER

Caso 1 — Menú principal

Terminal — tp_ejecutable

```
$ ./tp_ejecutable
*************** MENÚ PRINCIPAL ***************

1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo.
4. Corregir un registro del archivo.
5. Corregir un registro en memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Finalizar programa.

>
```

Caso 2 — Crear un archivo de ataque nuevo

Inicio de la carga: el despegue en (2, 10) con movimiento a (2, 9).

```
> 3

---------------- CREAR ATAQUE NUEVO ----------------

Decime la ruta completa del archivo del nuevo ataque:
> C:\Users\mauro\utn\ataque_prueba.dat
Indique donde despegara el dron:
x y > 2 10
Espera? (Ingrese tiempo en segundos):
0
Ingresa siguiente coordenada del dron:
x y > 2 9
Se guardó la orden con éxito!!

Resumen ataque nuevo:
El dron se movio 1 veces por el mapa.
Despega con exito.

1. Agregar orden.
0. Cancelar creación neuvo ataque.
Opcion > 1
x y > 2 9
Espera? (Ingrese tiempo en segundos):
5
Aterriza? (0 = NO, 1 = SI):0
Suelta granada 1? (0 = NO, 1 = SI):0
Suelta granada 2? (0 = NO, 1 = SI):0
Ataque kamikaze? (0 = NO, 1 = SI): 0
Ingresa siguiente coordenada del dron:
x y > 3 8
Se guardó la orden con éxito!!

Resumen ataque nuevo:
El dron se movio 2 veces por el mapa.
Despega con exito.
```

Final de la carga: el ataque kamikaze en (6, 6) cierra la ruta y el programa ofrece volver al menú.

```
Resumen ataque nuevo:
El dron se movio 5 veces por el mapa.
Despega con exito.
Soltó la granada 1.
Soltó la granada 2.

1. Agregar orden.
0. Cancelar creación neuvo ataque.
Opcion > 1
x y > 6 6
Espera? (Ingrese tiempo en segundos):
0
Aterriza? (0 = NO, 1 = SI):0
Ataque kamikaze? (0 = NO, 1 = SI): 1
Se guardó la orden con éxito!!

Resumen ataque nuevo:
El dron se movio 6 veces por el mapa.
Despega con exito.
Soltó la granada 1.
Soltó la granada 2.
Hizo ataque kamikaze.
Ataque finalizado y guardado con exito!!! volviendo al menu...

Presione ENTER para continuar...
```

Caso 3 — Cargar el archivo en memoria

Terminal — tp_ejecutable

```
> 1

-------------- CARGAR ATAQUE EN MEMORIA --------------

Decime la ruta completa del archivo de ataque:
> ataque_demo.dat
Archivo abierto correctamente.
Ataque cargado correctamente.

Presione ENTER para continuar...
```

Caso 4 — Mostrar el ataque cargado

Terminal — tp_ejecutable

```
> 2

--------------- MOSTRAR ATAQUE CARGADO ---------------

[X: 2, Y: 10] -> Acciones: Despegue | Siguiente: [2, 9]
[X: 2, Y: 9] -> Acciones: Esperar 2s | Siguiente: [3, 8]
[X: 3, Y: 8] -> Acciones: Soltar Granada 1 | Siguiente: [4, 8]
[X: 4, Y: 8] -> Acciones: Mover | Siguiente: [5, 7]
[X: 5, Y: 7] -> Acciones: Soltar Granada 2 | Siguiente: [6, 6]
[X: 6, Y: 6] -> Acciones: Ataque Kamikaze | FIN DE LA RUTA

Presione ENTER para continuar...
```

Caso 5 — Corregir un registro del archivo

Se agrega una espera de 3 s en (4, 8), que antes solo tenía movimiento.

Terminal — tp_ejecutable

```
> 4

--------------- CORREGIR REGISTRO DEL ARCHIVO ---------------

Decime la ruta completa del archivo de ataque:
> ataque_demo.dat
Ingrese la posicion X a corregir: 4
Ingrese la posicion Y a corregir: 8

Registro actual:
[X: 4, Y: 8] -> Acciones: Mover | Siguiente: [5, 7]

Ingrese los nuevos datos (1 = Si, 0 = No):
Despegue (actual: 0) (1/0): 0
Aterrizaje (actual: 0) (1/0): 0
Ataque Kamikaze (actual: 0) (1/0): 0
Soltar Granada 1 (actual: 0) (1/0): 0
Soltar Granada 2 (actual: 0) (1/0): 0
Tiempo de espera en segundos (actual: 0): 3
Siguiente X (actual: 5): 5
Siguiente Y (actual: 7): 7

Registro corregido correctamente.

Presione ENTER para continuar...
```

Después de volver a cargar el archivo con la operación 1, la operación 2 muestra la espera nueva en (4, 8).

Terminal — tp_ejecutable

```
> 2

--------------- MOSTRAR ATAQUE CARGADO ---------------

[X: 2, Y: 10] -> Acciones: Despegue | Siguiente: [2, 9]
[X: 2, Y: 9] -> Acciones: Esperar 2s | Siguiente: [3, 8]
[X: 3, Y: 8] -> Acciones: Soltar Granada 1 | Siguiente: [4, 8]
[X: 4, Y: 8] -> Acciones: Esperar 3s | Siguiente: [5, 7]
[X: 5, Y: 7] -> Acciones: Soltar Granada 2 | Siguiente: [6, 6]
[X: 6, Y: 6] -> Acciones: Ataque Kamikaze | FIN DE LA RUTA

Presione ENTER para continuar...
```

Caso 6 — Corregir una coordenada que no está en el archivo

Terminal — tp_ejecutable

```
> 4

--------------- CORREGIR REGISTRO DEL ARCHIVO ---------------

Decime la ruta completa del archivo de ataque:
> ataque_demo.dat
Ingrese la posicion X a corregir: 50
Ingrese la posicion Y a corregir: 50
No hay ningun registro en el archivo con esa coordenada.

Presione ENTER para continuar...
```

Caso 7 — Corregir un registro en memoria

La espera de (2, 9) pasa de 2 s a 5 s.

Terminal — tp_ejecutable

```
> 5

--------------- CORREGIR REGISTRO EN MEMORIA ---------------

Ingrese la posicion X a corregir: 2
Ingrese la posicion Y a corregir: 9

Registro actual:
[X: 2, Y: 9] -> Acciones: Esperar 2s | Siguiente: [3, 8]

Ingrese los nuevos datos (1 = Si, 0 = No):
Despegue (actual: 0) (1/0): 0
Aterrizaje (actual: 0) (1/0): 0
Ataque Kamikaze (actual: 0) (1/0): 0
Soltar Granada 1 (actual: 0) (1/0): 0
Soltar Granada 2 (actual: 0) (1/0): 0
Tiempo de espera en segundos (actual: 2): 5
Siguiente X (actual: 3): 3
Siguiente Y (actual: 8): 8

Registro corregido correctamente.

Presione ENTER para continuar...
```

Terminal — tp_ejecutable

```
> 2

--------------- MOSTRAR ATAQUE CARGADO ---------------

[X: 2, Y: 10] -> Acciones: Despegue | Siguiente: [2, 9]
[X: 2, Y: 9] -> Acciones: Esperar 5s | Siguiente: [3, 8]
[X: 3, Y: 8] -> Acciones: Soltar Granada 1 | Siguiente: [4, 8]
[X: 4, Y: 8] -> Acciones: Esperar 3s | Siguiente: [5, 7]
[X: 5, Y: 7] -> Acciones: Soltar Granada 2 | Siguiente: [6, 6]
[X: 6, Y: 6] -> Acciones: Ataque Kamikaze | FIN DE LA RUTA

Presione ENTER para continuar...
```

Caso 8 — Guardar la memoria en un archivo nuevo

Terminal — tp_ejecutable

```
> 6

---------------ingrese el nombre del nuevo archivo:
ataque_corregido.dat

---------------ingrese la ruta de la carpeta donde estará el archivo:
.

---------------Archivo cargado con éxito

Presione ENTER para continuar...
```

Caso 9 — Visualizar el ataque en HTML

La operación 7 no muestra nada en la terminal: crea `ataque_demo.html` junto al archivo cargado. Así se ve abierto en el navegador.

```
*************** MENÚ PRINCIPAL ***************

1. Cargar archivo de ataque en memoria.
2. Mostrar ataque cargado.
3. Crear un archivo de ataque nuevo.
4. Corregir un registro del archivo.
5. Corregir un registro en memoria.
6. Guardar memoria en un archivo nuevo.
7. Visualizar un archivo de ataque en html.
0. Finalizar programa.

> 7
Se creo el reporte HTML en la misma ruta del archivo: C:\Users\mauro\utn\ataque_demo.html

Presione ENTER para continuar...
```

Reporte ataque ataque_demo

N°	Coordenada		Despega	Aterriza	Suelta granada 1	Suelta granada 2	Ataque kamikaze	Espera	Se mueve a	
	X	Y							X	Y
1	2	10	SI	-	-	-	-	-	2	9
2	2	9	-	-	-	-	-	5 s.	3	8
3	3	8	-	-	SI	-	-	-	4	8
4	4	8	-	-	-	-	-	3 s.	5	7
5	5	7	-	-	-	SI	-	-	6	6
6	6	6	-	-	-	-	SI	-	-	-

Caso 10 — Archivo inválido

Con la operación 4 se marca (4, 8) como despegue, así `ataque_corregido.dat` queda con dos despegues.

Terminal — tp_ejecutable

```
> 4

--------------- CORREGIR REGISTRO DEL ARCHIVO ---------------

Decime la ruta completa del archivo de ataque:
> ataque_corregido.dat
Ingrese la posicion X a corregir: 4
Ingrese la posicion Y a corregir: 8

Registro actual:
[X: 4, Y: 8] -> Acciones: Esperar 3s | Siguiente: [5, 7]

Ingrese los nuevos datos (1 = Si, 0 = No):
Despegue (actual: 0) (1/0): 1
Tiempo de espera en segundos (actual: 3): 0
Siguiente X (actual: 5): 5
Siguiente Y (actual: 7): 7

Registro corregido correctamente.

Presione ENTER para continuar...
```

Al intentar cargarlo, la operación 1 lo rechaza y no carga nada en memoria.

Terminal — tp_ejecutable

```
> 1

--------------- CARGAR ATAQUE EN MEMORIA ---------------

Decime la ruta completa del archivo de ataque:
> ataque_corregido.dat
Archivo abierto correctamente.

---------------El archivo no cumple con los requisitos, intente con otro

---------------
Presione ENTER para continuar...
```

Caso 11 — Paginado al mostrar

La ruta de 20 órdenes, de (1, 1) a (1, 20), se creó con la operación 3 y se cargó con la operación 1. La operación 2 se detiene después de la orden 15 hasta que se presiona ENTER.

Terminal — tp_ejecutable

```
> 2

--------------- MOSTRAR ATAQUE CARGADO ---------------

[X: 1, Y: 1] -> Acciones: Despegue | Siguiente: [1, 2]
[X: 1, Y: 2] -> Acciones: Mover | Siguiente: [1, 3]
[X: 1, Y: 3] -> Acciones: Mover | Siguiente: [1, 4]
[X: 1, Y: 4] -> Acciones: Mover | Siguiente: [1, 5]
[X: 1, Y: 5] -> Acciones: Mover | Siguiente: [1, 6]
[X: 1, Y: 6] -> Acciones: Mover | Siguiente: [1, 7]
[X: 1, Y: 7] -> Acciones: Mover | Siguiente: [1, 8]
[X: 1, Y: 8] -> Acciones: Mover | Siguiente: [1, 9]
[X: 1, Y: 9] -> Acciones: Mover | Siguiente: [1, 10]
[X: 1, Y: 10] -> Acciones: Mover | Siguiente: [1, 11]
[X: 1, Y: 11] -> Acciones: Mover | Siguiente: [1, 12]
[X: 1, Y: 12] -> Acciones: Mover | Siguiente: [1, 13]
[X: 1, Y: 13] -> Acciones: Mover | Siguiente: [1, 14]
[X: 1, Y: 14] -> Acciones: Mover | Siguiente: [1, 15]
[X: 1, Y: 15] -> Acciones: Mover | Siguiente: [1, 16]

Presione ENTER para continuar...


[X: 1, Y: 16] -> Acciones: Mover | Siguiente: [1, 17]
[X: 1, Y: 17] -> Acciones: Mover | Siguiente: [1, 18]
[X: 1, Y: 18] -> Acciones: Mover | Siguiente: [1, 19]
[X: 1, Y: 19] -> Acciones: Mover | Siguiente: [1, 20]
[X: 1, Y: 20] -> Acciones: Aterrizaje | FIN DE LA RUTA

Presione ENTER para continuar...
```